15.780 Project Report: Optimizing Locations for U.S. Emergency Medical Resource Stockpiles for COVID-19

Kai Edick, Greta Reitenbach, and Connie Huang

1. Problem Statement and Importance

The COVID-19 pandemic was one of the most significant global public health crises in modern history, causing unprecedented strain on healthcare systems and exposing major gaps in resource allocation. While some regions faced critical shortages of resources, others had surplus capacity. The United States reportedly maintains twelve regional stockpiles of emergency medical supplies, but their exact locations and distribution strategies are not publicly disclosed.

Our project examines how these stockpiles could be optimally located and utilized to respond to county-level variations in COVID-19 case rates. Though the COVID-19 pandemic has subsided, we ran our project as if we were experiencing it currently from September 2022. Using historical data up to this point on infection levels, hospitalization rates, and the population distribution, we first estimate the COVID-19 rates for each county up to May 2023, which proxies how many resources each county would require during an outbreak. We then use an optimization model to determine the ideal placement of twelve stockpile centers that minimizes total travel distance weighted by predicted demand. This approach offers insight into how a national network of emergency supply hubs could be positioned to maximize accessibility and responsiveness.

Effective placement of emergency stockpiles can save lives by reducing delivery times and ensuring that supplies reach the hardest-hit regions first. During COVID-19, bottlenecks in distributing ventilators, testing kits, and personal protective equipment (PPE) greatly limited health responses, demonstrating the importance of advance planning and geographic coordination.

By combining predictive analytics with optimization, our project aims to design a framework for a more efficient medical supply distribution network. This model can support policymakers and emergency management agencies in evaluating current stockpile strategies and preparing for future public health crises. The insights extend beyond pandemics to other scenarios requiring rapid, large-scale deployment of resources such as natural disasters or bioterrorism responses.

2. Data

To be able to forecast future weekly COVID-19 case rates and then create an optimization model, we needed to aggregate data from various datasets. This would bring together key features that can help create a more accurate predictive model.

1. Our primary dataset, “Weekly United States COVID-19 Cases and Deaths by County,” is provided by the CDC and available to the public from the CDC website here. This dataset tracks weekly, county-level COVID-19 figures with about 566K data points from 01/22/2020 to 05/10/2023, so the dataset covers the spread, peak, and decline of the pandemic. We are provided with 9 features. For our purposes, we used the fields *fips_code* (a county’s unique FIPS ID code), *date*, and *new_cases*.
2. To supplement this, we referred to a Johns Hopkins University county-by-county demographic dataset “us_county” from Kaggle that utilizes 2014-2018 releases of the

American Community Survey (here). Out of the 11 provided features, we made use of the *fips*, *population*, and *median_age* fields to estimate the proportion of the county that is infected and measure a key risk factor, age.

3. Additionally, we used "National Counties Gazetteer" from the United States Census Bureau (here). Out of its 10 features, we used *GEOID* (maps to fips) and *ALAND_SQMI*, the land area of each county, to estimate population density.

These were good datasets to address the supply distribution problem because they contained detailed, reliable information about county-level COVID-19 case rates and physical county characteristics (population and land area), which would allow us to predict future COVID-19 rates. To combine these datasets, we used an inner join on the county FIPS ID code so that only counties appearing in all three datasets were retained. This ensured that each observation in our final dataset contained consistent information on weekly COVID-19 case rates, population size, median age, and land area. The resulting merged dataset provided a complete, county-level view that could be used both to forecast future case rates and to estimate the relative resource needs across counties.

The original data was well-maintained and good quality, and while there were no missing values in the dataset, any missing values would've been naturally handled when we combined the datasets with inner join. We kept the raw COVID-19 case data, since unusually high or low weekly values were realistic features of disease spread rather than data errors. However, we purposefully removed geographic "outliers" (counties in Alaska and Hawaii) to simplify our transportation assumptions, since their distances are not comparable to other counties and would distort the optimization model. The fields of our combined dataset are shown in *Figure 1*. Our final dataset had 549795 data points.

	Field	**Type**	**Description**	**Source Dataset**
1	*fips*	Text	FIPS code for individual counties	Exists across datasets, used to merge
2	*date*	Floating Timestamp	Date of report	Weekly United States COVID-19 Cases and Deaths by County (CDC)
3	*new_cases*	Number	New weekly cases reported	Weekly United States COVID-19 Cases and Deaths by County (CDC)
4	*population*	Number	Total population for the county	us_county (Johns Hopkins University)
5	*median_age*	Number	Overall median age for the county	us_county (Johns Hopkins University)
6	*ALAND_SQMI*	Number	Land area of each county	National Counties Gazetteer (US Census)

Figure 1: Original Data Organization

Data Processing

From the raw data, we created additional features to use in our analysis. These included calculations that capture meaningful county characteristics, as well as normalized variables to place all features on a comparable scale for modeling. Specifically, we constructed population density, weekly infection rate, and a spatial-lag measure of nearby infection rates, and then standardized these fields using z-score normalization (*Figure 2*).

One thing to note is we didn't use any PPE or ventilator quantity data, and instead used infection rates as a proxy for demand. For our future optimization, the objective only cares about relative demand.

	Field	**Type**	**Description**	**Formulation**
7	*pop_density*	Number	Population density of the county (people per sq. mi)	$\frac{population}{ALAND\_SQMI}$
8	*infection_rate*	Number	Weekly infection rate for the county	$\frac{new\_cases}{population}$
9	*spatial_lag*	Number	Distance-weighted average infection rate of nearby counties	Used Python library PySAL's *lag_spatial* function
10	*median_age_norm*	Number	Normalized median age of the county	Z-score normalization
11	*pop_density_norm*	Number	Normalized population density	Z-score normalization
12	*spatial_lag_norm*	Number	Normalized spatial lag	Z-score normalization

Figure 2: Processed Data Organization

3. Predictive Model

The goal of our predictive model is to predict the next week's infection rates in a given county. This supports the task of locating and allocating resources to the 12 national stockpiles since better predictive capabilities result in more efficient optimizations. Rather than optimizing stockpiles using hindsight, we aim to create a framework that operates under real-time uncertainty and can generalize to future disease outbreaks. To simulate this decision environment, we base predictions on recent infection trends and relevant exogenous factors, using them as the forward-looking inputs that would guide officials' allocation decision.

Given that we are making predictions based on time-series data in an autoregressive fashion, our group first considered the autoregressive, ARIMA-based models discussed in class. We ultimately settled on the SARIMAX model, then compared it to a random forest, neural network, and linear regression. SARIMAX is a more general version of the classic ARIMA model that includes exogenous variables and a seasonality component, which we believe are important to

control for known factors that affect disease spread. In particular, the county's median age, population density, and the weighted average infection rate of nearby counties (spatial lag) are known to affect disease spread, so we included them as exogenous variables, while we set the seasonality component to cover a 52-week (yearly) window to account for the effects of holidays and weather. As a side note, the order and seasonal order in the SARIMAX model were both set to (0, 1, 1) after looking at the autocorrelation plots, the exogenous variables were standardized using z-scores, and we used an 80/20 train/test split corresponding to the date 09/21/2022.

The random forest, neural network, and linear regression were used to compare against the SARIMAX model because they are common, general, benchmark models. To account for the autoregressive aspect of our prediction task, we included the county's infection rates for the past four weeks as additional variables. Therefore, note that all data prior to the fourth week is excluded (this is reasonable since most counties had little to no infection during this time).

The RMSE and MAE of each model type on the test dataset are listed in *Figure 3*. While the SARIMAX model had the lowest RMSE, it had a significantly larger MAE than the others. Furthermore, looking at *Figures 4-7*, its shape is visually inaccurate, especially on the test data. Therefore, we settled on the random forest model since it is otherwise the best performing model, and has the additional advantage of interpretability.

We believe the obtained results of the random forest are accurate and make sense because they track the true infection rate very closely, as can be seen from a visual inspection of *Figure 5*. The other models are not nearly as close to the true infection rates as the random forest. The neural network and linear regression models predicted flatter, smoother curves than the true ones, likely because they learned to ignore a lot of the variation, leading to its reasonable RMSE and MAE. *Figures 4-7* use data for Los Angeles County, which has a large population so there is a central-limit-theorem-like effect that results in a smoother true curve, but the models are also trained on smaller counties that tend to have much larger variations. Hence, the neural network and linear regression models learned to stick near the mean, but the random forest still performed well on these noisy counties. Therefore, as can be seen in *Figures 6* and *7*, the neural network and linear regression models poorly predict peaks, which we want to avoid at all costs since lives are at stake if the infection rate is terribly underpredicted.

The SARIMAX model underperformed compared to our expectations on the test data. As seen in *Figure 4*, the curve predicted using test data peaks around February 2023. This is quite far off from reality, but looking historically, there are also peaks in February 2021 and 2022. Thus, the model likely learned from the seasonality component that there should be another peak in February 2023. We hypothesize that the earlier peaks are due to wintery conditions and holiday gatherings, but there was no peak in February 2023 due to high vaccination rates. Hence, incorporating vaccination rates is an exogenous variable to consider adding for future work. In conclusion, despite the SARIMAX's low RMSE and good fit on training data, we prefer the random forest model because of its excellent visual fit and generalizability to the test data and many counties.

RMSE and MAE on Test Data		
	RMSE	MAE

RMSE and MAE on Test Data		
SARIMAX	0.0027	0.0021
Random Forest	0.0050	0.0004
Neural Network	0.0051	0.0006
Linear Regression	0.0050	0.0005

Figure 3: RMSE and MAE

Note: A new SARIMAX model must be used for each county, therefore the RMSE and MAE reported above are the average RMSE and MAE of 20 randomly selected counties, due to computational constraints.

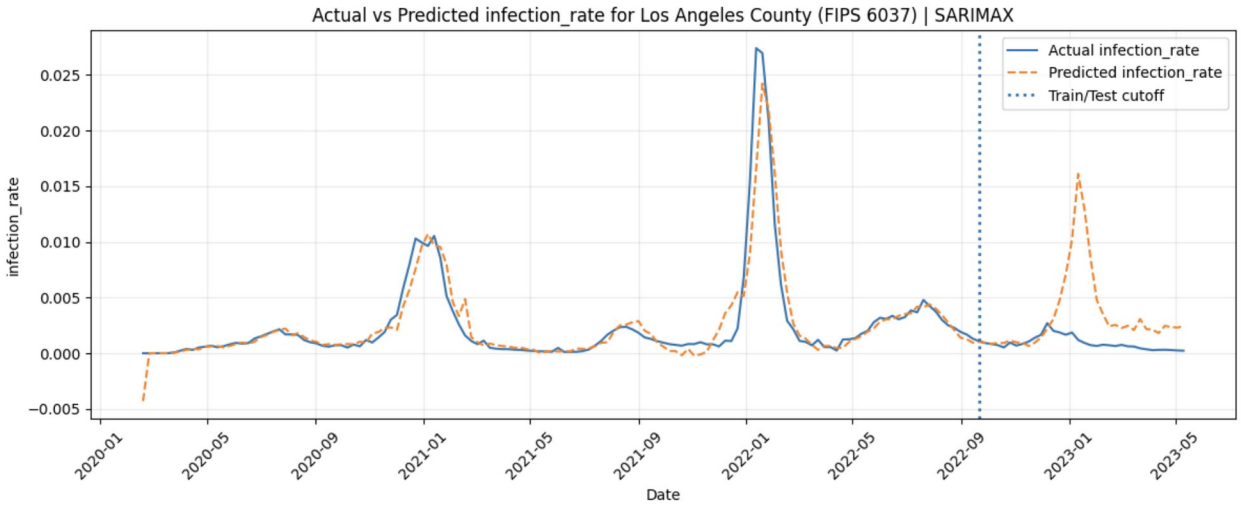

Figure 4: SARIMAX True vs. Predicted Infection Rate

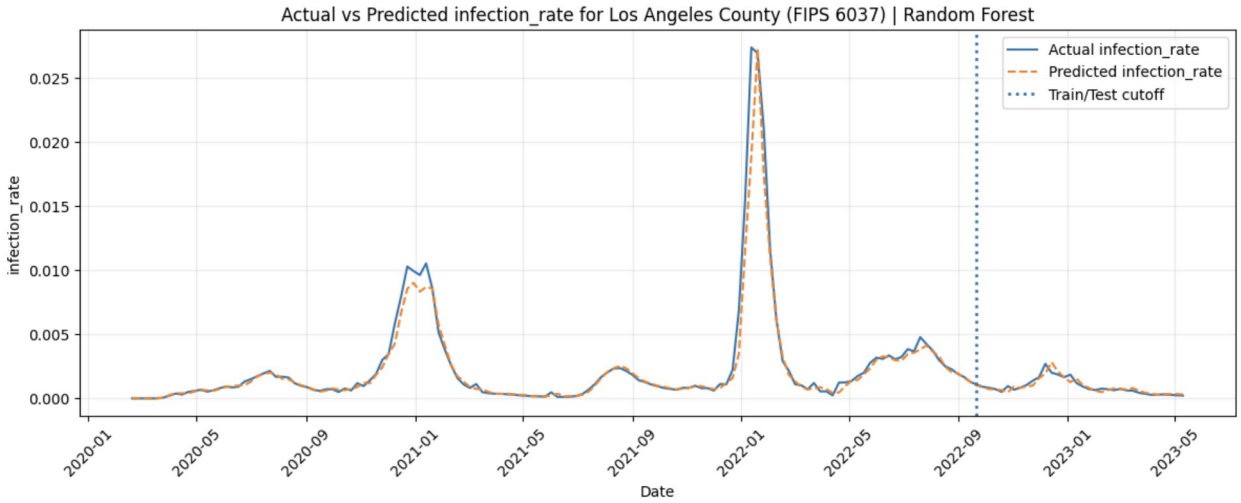

Figure 5: Random Forest True vs. Predicted Infection Rate

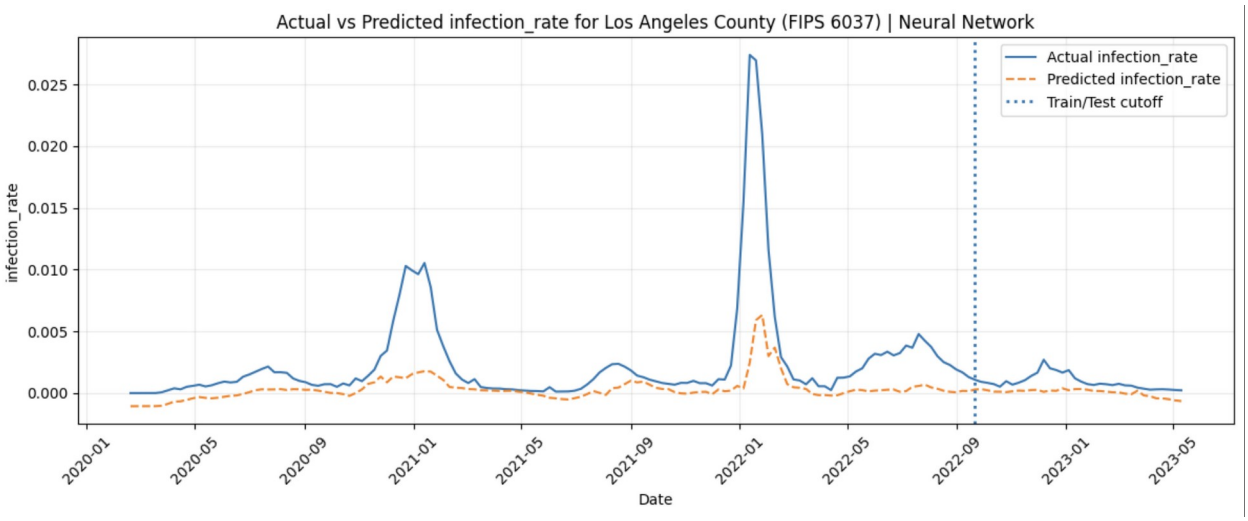

Figure 6: Neural Network True vs. Predicted Infection Rate

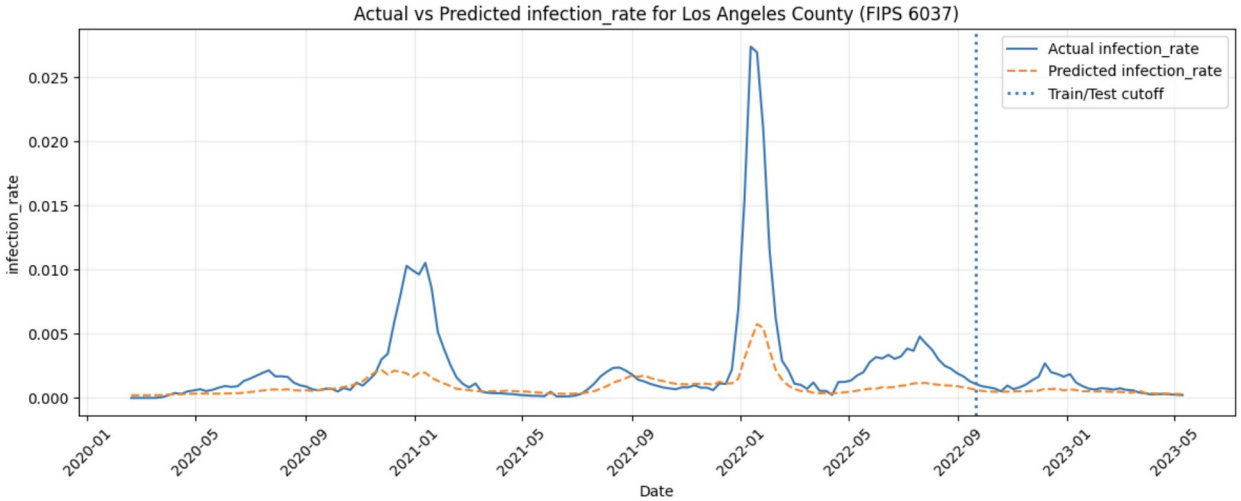

Figure 7: Neural Network True vs. Predicted Infection Rate

Feature Importances from Random Forest Regression	
infection_rate_lag1	0.667
infection_rate_lag2	0.031
infection_rate_lag3	0.031
infection_rate_lag4	0.028
spatial_lag	0.183
pop_density	0.045
median_age	0.016

Figure 8: Random Forest Feature Importances

This table shows the relative importances of each feature in the random forest regression. Importance is based on the effectiveness of that feature in properly splitting the data, so larger values indicate a more "impactful" feature. Clearly, the most significant feature was the infection rate in the previous week (lagX means the infection rate from X weeks ago), followed distantly by spatial_lag, while the remaining features appear less impactful.

4. Optimization Problem

To optimize the strategic national stockpile locations, we solved a capacitated facility location problem with a rolling horizon. This allowed us to determine the optimal placement of 12 emergency stockpile centers to minimize travel distance while meeting the fluctuating COVID-19 demand. Unlike a static model, we used our forecasted demand week-by-week to adjust inventory and shipments dynamically, which allowed the model to correct for prediction errors in future periods. In each step, we optimized flows based on predicted demand plus an adjustment term derived from the previous week's error (if actual demand exceeded prediction). This problem structures a national supply chain network, linking candidate stockpile locations (United States counties) to demand nodes (United States counties) to ensure responsiveness to volatile infection rates.

Decision Variables

a. Stockpile Location (X_i): a binary indicator variable.
    i. $X_i = 1$ if a stockpile is located in county i.
    ii. $X_i = 0$ otherwise.
b. $flow_{ij}$: a continuous, real-valued variable representing the flow of resources.
    i. It measures the quantity of supplies transported from stockpile county i to demand county j.
c. $unmet_j$: a continuous variable representing the amount of demand at county j that could not be satisfied by available inventory or new shipments.

Objective Function

Our objective function minimized a weighted sum of transportation costs and unmet demand penalties.

d. The travel distance d_{ij} was taken from the NBER pairwise county distance dataset and multiplied by $flow_{ij}$ from above. We penalized distance to optimize the reactive capability of this supply chain and prevent unrealistic cross-country shipments.
    i. Rather than calculate a specific travel cost between all pairwise county groups, we assume for simplicity that cost is a linear multiple of distance.

e. A penalty coefficient of 10,000 was applied to every unit of unmet demand, prioritizing fulfillment over transport distance. Unmet demand was calculated from the difference between total inventory and shipment and realized demand.

$$\min Z = \sum_{i \in stockpile\ counties} \sum_{j \in counties} (d_{ij} \times flow_{ij}) + 10000 \times \sum_{j \in counties} unmet_j$$

Constraints

f. Cardinality Constraint: There are exactly 12 stockpiles. In practice, the United States Strategic National Stockpile maintains 12 sites across the country. The location of these sites are hidden for national security purposes.

$$\sum_{i} X_i = 12$$

g. Global Capacity Constraint: The total flow of supplies across all weeks cannot pass $S = 66$ million. We measure the amount of supplies in terms of the number of people supported, i.e. if $S = 10$ then there is only enough supplies for 10 people. This number is taken from the Strategic National Stockpile planning target for population coverage during a severe infectious disease event. Because of this, we assume for simplicity that one unit of stock corresponds to one infectious case treated.

$$\sum_{i,j} flow_{ij} \leq S$$

h. Linking Constraint: Flows from a county *i* are only allowed if that county is selected as a stockpile. That flow is capped by the global capacity scaled by the binary variable (analogously to the big-M constraint).

$$\sum_{j} flow_{ij} \leq S \times X_i$$

i. Demand Satisfaction Constraint: For each county *j*, the sum of inflows plus previous inventory plus the unmet variable must meet the net demand. Inventory is calculated as the remainder of the original total stockpile available (decreasing across weeks).

$$\sum_{i} flow_{ij} + unmet_j \geq Demand_j - Inventory_j$$ (applied only if Demand > Inventory)

Model Formulation Implications

1. We intentionally set the unmet demand penalty to 10,000, significantly higher than typical distances, to ensure that meeting demand, and therefore saving lives, is vastly more important than saving money on transportation costs.
2. Because we subtracted total shipped goods in every weekly loop, our results show how a finite resource is depleted over time. If continued for an extended period of time, the model would highlight when the national stockpile would run dry.

3. Our supply chain is reactive. If demand was under-predicted in one week, we “made up for it” in the next week by increasing the targeted demand, preventing a cascading failure of critical resource shortages.

5. Results

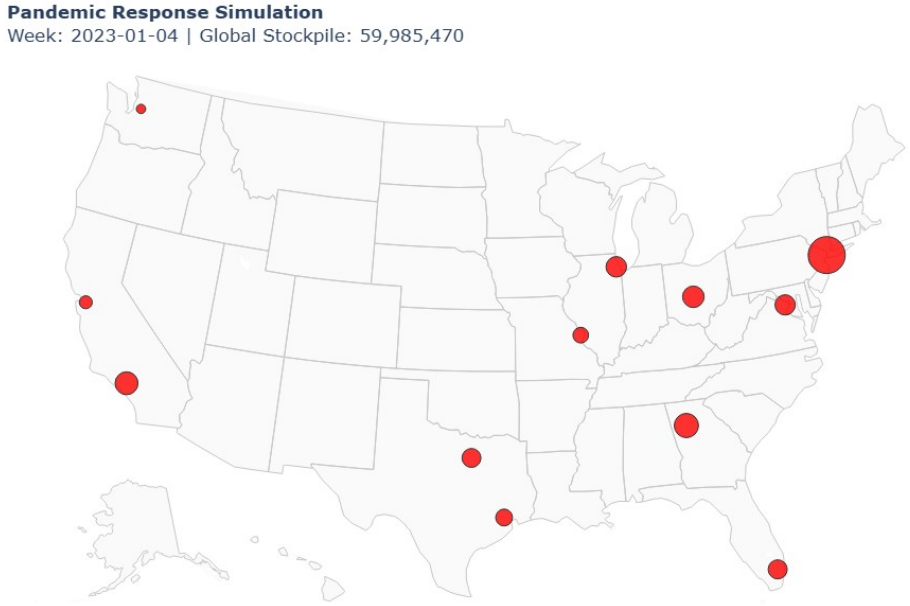

Figure 5: Example snapshot of resource distribution, shown on January 4, 2023

date	total_cost	total_shipped	unmet_demand	stock_remaining	stockpile_sites
2022-09-21	68343992.61	415819	0	65584181	[6037, 17031, 48
2022-09-28	58074385.52	388936	0	65195245	[6037, 17031, 48
2022-10-05	50434391.46	325119	0	64870126	[6037, 17031, 48
2022-10-12	48993956.57	302471	0	64567655	[6037, 17031, 36
2022-10-19	42636727.59	266370	0	64301285	[6037, 17031, 48
2022-10-26	46708268.48	279047	0	64022238	[17031, 48113, 6
2022-11-02	46934738.2	288878	0	63733360	[6037, 17031, 48
2022-11-09	51274481.41	306530	0	63426830	[6037, 17031, 48
2022-11-16	66026602.08	331931	0	63094899	[6037, 17031, 40
2022-11-23	49805853.66	308154	0	62786745	[6037, 17031, 40
2022-11-30	55881439.42	350256	0	62436489	[6037, 17031, 40
2022-12-07	58369451.67	376694	0	62059795	[6037, 17031, 40
2022-12-14	90922056.08	597945	0	61461850	[6037, 17031, 40
2022-12-21	76670938.44	498693	0	60963157	[6037, 17031, 40
2022-12-28	88343265.38	566701	0	60396456	[17031, 48201, 6
2023-01-04	62924131.97	410986	0	59985470	[6037, 17031, 48
2023-01-11	80127881.23	549962	0	59435508	[6037, 17031, 48
2023-01-18	65312479.85	438967	0	58996541	[6037, 17031, 48
2023-01-25	55570993.99	334146	0	58662395	[6037, 17031, 48
2023-02-01	47695671.72	294885	0	58367510	[6037, 17031, 48
2023-02-08	49228864.69	298642	0	58068868	[6037, 17031, 48
2023-02-15	47680888.55	292163	0	57776705	[6037, 17031, 48
2023-02-22	44725867.58	273156	0	57503549	[6037, 17031, 48
2023-03-01	41547604.98	290055	0	57213494	[6037, 17031, 40
2023-03-08	40451744.86	252011	0	56961483	[6037, 17031, 40
2023-03-15	31692863.23	195326	0	56766157	[6037, 17031, 48
2023-03-22	36124240.89	206381	0	56559776	[6037, 17031, 40
2023-03-29	27972001.22	170135	0	56389641	[6037, 17031, 48
2023-04-05	27737905.35	166967	0	56222674	[6037, 17031, 48
2023-04-12	24508397.29	145041	0	56077633	[6037, 17031, 48
2023-04-19	19664383.31	116798	0	55960835	[6037, 17031, 40
2023-04-26	20629323.31	115731	0	55845104	[6037, 17031, 48
2023-05-03	19564908.99	111474	0	55733630	[6037, 17031, 48
2023-05-10	17749383.63	97225	0	55636405	[17031, 4013, 60

Figure 6:Optimization results showing weekly resource allocation

Our optimization model was executed over a test period from September 21, 2022 through May 17, 2023. The simulation successfully allocated resources from a starting global stockpile of 66 million units, dynamically adjusting to weekly demand fluctuations.

1. The model effectively managed the finite global stock over the simulation horizon. As shown in the data, the “stock_remaining” decreased steadily from the initial 66 million. The system experienced its highest stress period in mid-December. The week of December 14, 2022 saw the highest total shipment volume of 597,945 units. This correlates to the winter surge of COVID-19 cases often observed in historical data. Transportation costs peaked alongside shipment volume, reaching approximately 90.9 million (distance-weighted units) during the same week of December 14. By the end of the simulation in May 2023, the stockpile had approximately 55.6 million units remaining. This indicates that the strategic reserve of 66 million was sufficient to withstand the forecasted demand waves without total depletion, validating the adequacy of the initial supply cap for this specific timeline. This indicates that almost two years into the pandemic, the biggest issue in the United States’ response was not the number of units stockpiled, but the distribution of these supplies across the country.
2. One key finding of the optimization was the stability of the selected stockpile locations. While the model was free to choose any twelve sites from the list each week, the optimal set remained extremely consistent. Certain FIPS codes appeared in the optimal set nearly every week of the simulation. For example, FIPS codes 6037 (Los Angeles County, CA), 17031 (Cook County / Chicago, IL), and 48201 (Harris County / Houston, TX) served as anchors for the response network. Figure 5 above visually confirms this finding. It displays a clear geographic spread with hubs in the follow areas:
    i. West Coast: covering California and the Pacific Northwest.
    ii. Northeast: a large node covering the dense New York and New England region.
    iii. Midwest: centered around Chicago.
    iv. South: large nodes in Texas and Florida.

    This stability suggests that a static network design, rather than one that spends money to move stockpiles weekly, would likely be highly effective. The optimal locations are driven heavily by population density centers, which do not change rapidly.
3. One critical success metric for the model was its ability to meet demand. Throughout the reported weeks from September 2022 to May 2023, the unmet demand remained at zero. This demonstrates that the transportation network and inventory levels were sufficient to cover 100% of the predicted infection needs across all counties. The high penalty for unmet demand in the objective function successfully forced the solver to prioritize fulfillment over transportation savings.

The results indicate that a network of 12 strategically placed national stockpiles, anchored in major metropolitan areas (including Los Angeles, Chicago, Houston, and New York), is sufficient to meet demand during a typical infection wave. The consistent selection of these sites suggests that local and national officials should focus on permanent infrastructure in these hubs rather than temporary, mobile solutions.

6. Conclusion

From our prediction and optimization models, we hope to demonstrate how data can guide the national medical supply distribution to most effectively reach people who need it. Going forward, public health agencies and emergency management organizations can use this

framework to support planning for future outbreaks or large-scale emergencies. Specifically, our process enables organizations to identify cost-efficient stockpile locations based on real disease dynamics, quantify transportation and distribution costs to enable more accurate budgeting for response efforts, allocate medical supplies proportionally to county-level risk, and stress-test logistical systems by running the model under different constraints.

Extensions of this project could incorporate real transportation networks, as our model currently uses distance-based approximations without considering real travel times. Future work could also integrate demographic indicators, including socioeconomic status and accessibility challenges, to promote more equitable allocation decisions. As noted in Section 3, vaccination rates could also be incorporated to improve the predictive model's capabilities.

It would also be interesting to see if dynamic stockpile locations might be more effective than relying on static stockpile placements. Future research could evaluate the costs and benefits of temporary or seasonal relocations instead of keeping fixed locations. Our project can also be brought in different directions. Beyond pandemics, the same approach can be applied to deciding strategic reserves of equipment for situations where rapid deployment is critical, like wildfire response kits and hurricane relief resources.

7. Appendix A: SARIMAX

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
import random
import os
import warnings

warnings.filterwarnings("ignore")

if not os.path.exists('county_forecasts'):
    os.makedirs('county_forecasts')
def run_county_forecast(df, target_fips):
    county_data = df[df['fips'] == target_fips].copy()
    county_data = county_data.sort_values('date')
    county_data.set_index('date', inplace=True)

    target_col = 'infection_rate'

    exog_cols = ['median_age_norm', 'pop_density_norm', 'spatial_lag_norm']

    train = county_data.loc[:'2021-12-31']
    test = county_data.loc['2022-01-01':'2022-12-31']

    if len(train) < 10 or len(test) < 1:
        raise ValueError("Insufficient data")

    model = sm.tsa.statespace.SARIMAX(
        train[target_col],
        exog=train[exog_cols],
        order=(0, 1, 1),
        seasonal_order=(0, 1, 1, 52),
        enforce_stationarity=False,
        enforce_invertibility=False
    )

    results = model.fit(disp=False)

    forecast_res = results.get_forecast(steps=len(test), exog=test[exog_cols])
    pred_mean = forecast_res.predicted_mean.clip(lower=0)

    conf_int = forecast_res.conf_int()
    conf_int[conf_int < 0] = 0

    rmse = np.sqrt(mean_squared_error(test[target_col], pred_mean))
    return rmse

file_path = 'processed_covid_data.csv'
df = pd.read_csv(file_path)
df['date'] = pd.to_datetime(df['date'])

unique_fips = df['fips'].unique()
random_fips = random.sample(list(unique_fips), 5)

for fips in random_fips:
    try:
        rmse_val = run_county_forecast(df, fips)
    except Exception as e:
```

```
        print(f"Failed ({str(e)})")
```

8. Appendix B: Random Forest

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor

file_path = 'processed_covid_data.csv'
df = pd.read_csv(file_path)

df['date'] = pd.to_datetime(df['date'])
df = df.sort_values(['fips', 'date']).reset_index(drop=True)

def add_lags(data, group_col, target_cols, lags):
    data = data.copy()
    lagged_cols = []
    for col in target_cols:
        for lag in lags:
            lag_col_name = f"{col}_lag{lag}"
            data[lag_col_name] = (
                data.groupby(group_col)[col]
                .shift(lag)
            )
            lagged_cols.append(lag_col_name)
    return data, lagged_cols

lagged_targets = ['infection_rate']
lags = [1, 2, 3, 4]

df_lagged, lagged_columns = add_lags(df, group_col='fips', target_cols=lagged_targets, lags=lags)

df_model = df_lagged.dropna().reset_index(drop=True)

target_col = 'infection_rate'
feature_cols = [
    'median_age_norm',
    'pop_density_norm',
    'spatial_lag_norm',
] + lagged_columns

unique_dates = np.sort(df_model['date'].unique())
cutoff_index = int(0.8 * len(unique_dates))
cutoff_date = unique_dates[cutoff_index]

train_mask = df_model['date'] < cutoff_date
test_mask = df_model['date'] >= cutoff_date

X_train = df_model.loc[train_mask, feature_cols]
y_train = df_model.loc[train_mask, target_col]

X_all = df_model[feature_cols]

rf = RandomForestRegressor(
    n_estimators=300,
    min_samples_leaf=5,
    n_jobs=-1,
    random_state=24
)
```

```
rf.fit(X_train, y_train)
df_model['predicted_rate'] = rf.predict(X_all)

df_model['split'] = np.where(df_model['date'] < cutoff_date, 'train', 'test')

output_df = df_model[['fips', 'date', 'infection_rate', 'predicted_rate', 'split']]
output_df.rename(columns={'infection_rate': 'actual_rate'}, inplace=True)

output_filename = 'county_infection_predictions.csv'
output_df.to_csv(output_filename, index=False)
```

9. Appendix C: Optimization Script

```
import pandas as pd
import numpy as np
import gurobipy as gp
from gurobipy import GRB
import json

# Data loading
pred_df = pd.read_csv('county_infection_predictions.csv')
pred_df['date'] = pd.to_datetime(pred_df['date'])
pred_df['fips'] = pd.to_numeric(pred_df['fips'], errors='coerce').fillna(0).astype(int)

pop_source = pd.read_csv('processed_covid_data.csv')
pop_source['fips'] = pd.to_numeric(pop_source['fips'], errors='coerce').fillna(0).astype(int)
pop_df = pop_source[['fips', 'population']].drop_duplicates(subset='fips').set_index('fips')

df = pred_df.merge(pop_df, on='fips', how='left')
df['population'] = df['population'].fillna(df['population'].median())
df['predicted_demand'] = (df['predicted_rate'] * df['population']).fillna(0)
df['actual_demand'] = (df['actual_rate'] * df['population']).fillna(0)

test_df = df[df['split'] == 'test'].copy()
test_weeks = np.sort(test_df['date'].unique())

candidate_fips = pop_df.nlargest(50, 'population').index.astype(int).tolist()
all_counties = sorted(df['fips'].unique().astype(int).tolist())

def load_distances(filepath):
    dist_map = {}
    d_df = pd.read_csv(filepath)
    d_df.columns = [c.replace('"', '').strip() for c in d_df.columns]
    col_a, col_b, col_dist = 'county1', 'county2', 'mi_to_county'
    mask = (d_df[col_a].isin(candidate_fips)) & (d_df[col_b].isin(all_counties))
    for _, row in d_df[mask].iterrows():
        dist_map[int(row[col_a]), int(row[col_b])] = float(row[col_dist])

    for c in candidate_fips: dist_map[c, c] = 0.0
    return dist_map

distances = load_distances('data/nber_pairwise_distance.csv')

# Optimization function
def solve_week(week_date, candidates, counties, demand_map, distance_map, prev_inv, S_cap):
    m = gp.Model(f"Stockpile_{week_date}")
    m.setParam('OutputFlag', 0)

    # Variables
    x = m.addVars(candidates, vtype=GRB.BINARY, name="Open")
```

```
    flow = m.addVars(candidates, counties, vtype=GRB.CONTINUOUS, name="Flow")
    unmet = m.addVars(counties, vtype=GRB.CONTINUOUS, name="Unmet")

    # Objective
    obj_trans = gp.quicksum(distance_map.get((i,j), 3000) * flow[i,j] for i in candidates for j
in counties)
    obj_unmet = 10000 * unmet.sum()
    m.setObjective(obj_trans + obj_unmet, GRB.MINIMIZE)

    # Constraints
    m.addConstr(x.sum() == 12, "Limit_12")
    m.addConstr(flow.sum() <= S_cap, "GlobalCap")

    for i in candidates:
        m.addConstr(flow.sum(i, '*') <= S_cap * x[i], f"Link_{i}")

    for j in counties:
        dem = demand_map.get(j, 0)
        inv = prev_inv.get(j, 0)
        if dem > inv:
            m.addConstr(flow.sum('*', j) + unmet[j] >= dem - inv, f"Dem_{j}")

    m.optimize()

    if m.status == GRB.OPTIMAL:
        selected = [i for i in candidates if x[i].x > 0.5]

        stockpile_throughput = {}
        for i in selected:
            total_out = sum(flow[i,j].x for j in counties)
            stockpile_throughput[i] = int(total_out)

        new_inv = {}
        total_unmet = 0
        for j in counties:
            inflow = sum(flow[i,j].x for i in candidates)
            dem = demand_map.get(j, 0)
            inv = prev_inv.get(j, 0)
            consumed = min(inv + inflow, dem)
            new_inv[j] = (inv + inflow) - consumed
            total_unmet += unmet[j].x

        return selected, stockpile_throughput, new_inv, total_unmet, m.objVal
    else:
        return [], {}, prev_inv, 0, 0

TOTAL_STOCKPILE = 66000000
current_stock = TOTAL_STOCKPILE
current_inv = {f: 0 for f in all_counties}
demand_adj = {f: 0 for f in all_counties}
results = []

for date in test_weeks:
    date_str = pd.to_datetime(date).strftime('%Y-%m-%d')

    if current_stock <= 0:
        results.append({'date': date_str, 'cost': 0, 'unmet': 0, 'stock_remaining': 0,
'stockpile_flows': "{}"})
        continue
```

```
    weekly_grp = test_df[test_df['date'] == date].groupby('fips')[['predicted_demand',
'actual_demand']].sum()
    curr_dem = {}
    for f in all_counties:
        base = weekly_grp['predicted_demand'].get(f, 0)
        curr_dem[f] = float(base) + float(demand_adj.get(f, 0))

    sites, site_flows, new_inv, unmet, cost = solve_week(
        date_str, candidate_fips, all_counties, curr_dem, distances, current_inv, current_stock
    )

    total_shipped = sum(site_flows.values())
    current_stock -= total_shipped

    for f in all_counties:
        actual = float(weekly_grp['actual_demand'].get(f, 0))
        if curr_dem[f] < actual:
            demand_adj[f] = actual - curr_dem[f]
        else:
            demand_adj[f] = 0

    current_inv = new_inv

    results.append({
        'date': date_str,
        'total_cost': cost,
        'total_shipped': total_shipped,
        'unmet_demand': unmet,
        'stock_remaining': current_stock,
        'stockpile_sites': str(sites),
        'stockpile_flows': json.dumps(site_flows)
    })

pd.DataFrame(results).to_csv('stockpile_optimization_results.csv', index=False)
```